

# The *Custody Problem* in Enterprise Agentic AI

Every enterprise adopting third-party agentic platforms is quietly transferring custody of its operational reasoning to a runtime it does not own. Privacy promises don't address this. Architecture has to.

**What this document argues:** when an enterprise runs its work through a managed agent, it is no longer just buying model access – it is delegating context assembly, memory, tool execution, and policy defaults to that provider's runtime. This is a structural problem, not a security checkbox. This paper introduces a proposed open framework called **Hermetic Knowledge Isolation (HKI)** to give enterprises an enforceable runtime contract over what their agents can see, remember, and do.

- Runtime sovereignty
- Third-party agent risk
- Domain-scoped reasoning
- Explicit publication

AUTHOR  
**Henok Ghebrechristos, PhD**

PROPOSES  
**HKI – an open framework**

AUDIENCE  
**AI platform, security, architecture leads**



# The security boundary has moved from training data to the agentic runtime itself.

No-training guarantees answer an important question. Off-the-shelf enterprise agents add a second one: who controls the runtime that assembles knowledge, invokes tools, stores memory, and decides what the agent can do?

A provider can truthfully say, "We do not train on your enterprise data," while its managed agent still retrieves enterprise documents, injects internal context, calls tools, stores memory, routes requests, logs telemetry, and learns operational preferences during inference.

That is the risk enterprises accept when they adopt managed agentic platforms such as Gemini Enterprise, ChatGPT Enterprise, or similar bundled agents out of the box. The platform is no longer just a model endpoint. It becomes a privileged execution layer inside the enterprise's work.

Context bleed remains dangerous, but it is downstream of custody: the runtime may decide what it can see before the enterprise sets an operation-level boundary. HKI does not block legitimate cross-domain work; executive legal/logistics answers or pharmacy order lookups are composed from separately scoped operations, then joined through publication, handoff, or audited provenance.



# The custody problem appears when the agent runtime becomes a privileged black box.

The UI can look scoped while the execution path underneath quietly broadens through managed connectors, caches, graph traversal, accumulated memory, and tool routing.

01

## Third-party agent as vulnerability

The managed agent is granted context, memory, connectors, and tool authority. If custody is external, the agent becomes part of the attack surface – not through a breach, but through design.

02

## Context bleed

Retrieval, caches, graph edges, files, and conversations can inject adjacent-domain knowledge unless visibility is enforced at every transformation – not only at the final read.

03

## Memory and tool drift

Agent memory, tool routing, logs, and workflow preferences become hidden operational knowledge without durable labels – invisible to the enterprise that created them.

04

## Proof gap

Security claims remain promises unless every boundary is testable through conformance cases, adversarial probes, and verifiable release evidence at each runtime surface.

# The agent platform, not just the model, is the dependency.

Enterprise buyers often think they are purchasing model access. In practice, they are depending on a managed agentic substrate: memory, tool routing, retrieval, permissions, evaluation, logs, and workflow execution.

## THE AGENTIC REASONING SURFACE

### Retrieval

Documents, chunks, graph edges, indexes, and vector filters. Shapes every evidence set the model reasons from.

---

**RISK: SCOPE-UNAWARE RECALL RETURNS CROSS-DOMAIN CHUNKS**

### Memory

Preferences, vocabulary, escalation paths, and learned operational patterns accumulated across sessions.

---

**RISK: UNLABELED MEMORY CARRIES DOMAIN CONTEXT FORWARD INDEFINITELY**

### Tools

Actions, connector routing, privileges, and workflow execution habits built up over time.

---

**RISK: TOOL AUTHORITY GRANTED IN ONE DOMAIN APPLIES SILENTLY TO OTHERS**

### Cache

Prior answers, semantic similarity hits, and query execution traces stored for performance.

---

**RISK: DOMAIN-UNSCOPED KEYS ALLOW CROSS-DOMAIN ANSWER COLLISION**

### Telemetry

Failures, retries, latency tradeoffs, and operational patterns that expose strategy and priorities.

---

**RISK: OPERATIONAL SIGNAL REVEALS DOMAIN BEHAVIOUR TO PLATFORM OBSERVERS**

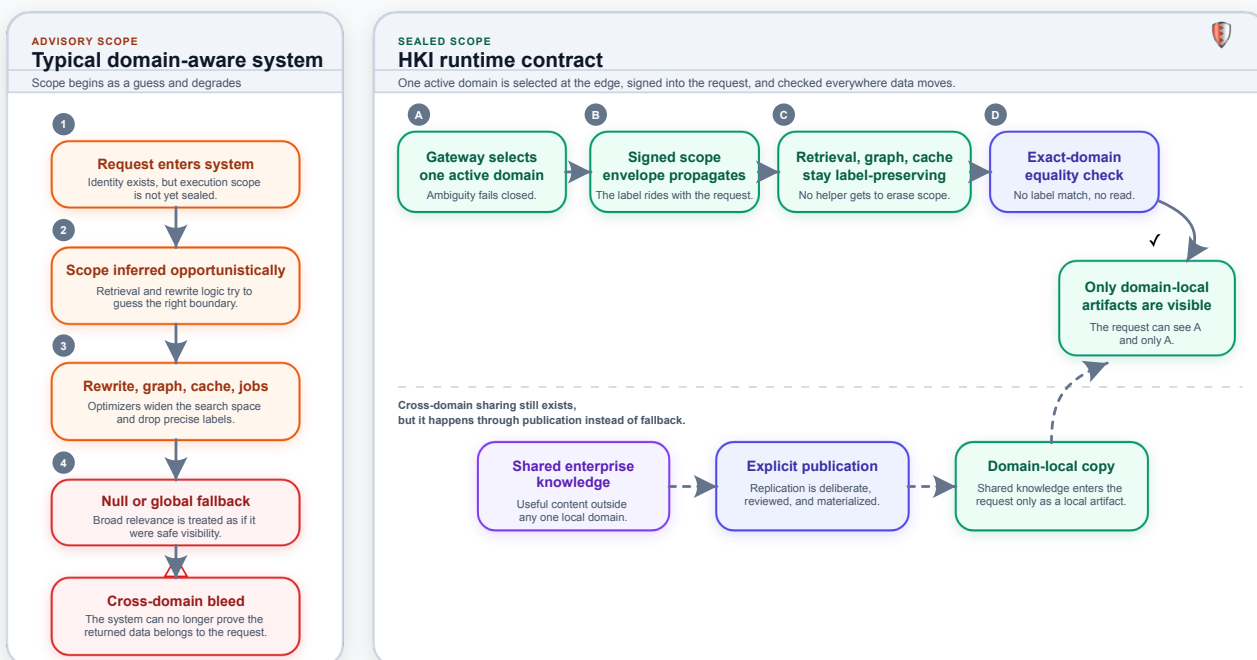
None of these layers has to become model-training data to become strategic signal. Whoever controls the agentic runtime controls what the agent sees, remembers, and does.

# Conventional scope drifts. HKI treats scope as a sealed execution label.

The contrast is simple: advisory scope starts as an intention and can erode. Runtime custody keeps the active domain attached to every operation.

## The flow contrast at a glance

Conventional systems often weaken scope as they optimize.  
HKI preserves one active domain from the gateway to every downstream decision.



HKI selects one active domain for each runtime operation, propagates a signed scope envelope, enforces exact-domain equality, and limits visibility to domain-local artifacts plus explicitly published copies.

# Switching costs grow quietly. By the time they are obvious, exit may be theoretical.

In cloud, dependency was compute and managed services. In agentic AI, dependency is reasoning, memory, and workflow execution.

## YEAR 0

### Off-the-shelf agent

Easy to adopt. The provider looks like a productivity vendor with familiar chat interfaces and low friction.

Connectors, memory, and retrieval defaults are set by the platform. The enterprise has not yet noticed what it has delegated – or to whom.

## YEAR 1

### Workflows accrue

Prompts, retrieval indexes, memories, and fine-tunes accumulate inside the managed runtime.

Custom instructions encode institutional knowledge. Leaving now means rebuilding months of domain-specific tuning in a new environment.

## YEAR 2

### Agents embed

Agents are inside daily operations. Exit now means re-engineering workflows and retraining entire teams.

Tool routing, approval chains, and knowledge bases live in the provider's runtime. Institutional intelligence is now an asset the enterprise does not own.

## YEAR 4

### Runtime owns memory

Switching is technically possible, but operationally painful to the point of being effectively locked in.

Terms are renegotiated from a position of dependency. The provider's pricing power reflects the cost of the operational intelligence it holds – not the compute it runs.

# The danger does not require villains. It requires a privileged runtime you do not control.

Wrong hands is less a movie scene than an ordinary platform reality: a managed agent with connector reach, memory, tool authority, pricing power, product withdrawals, policy shifts, and silent defaults.

## THE TECHNOFEUDAL ENDGAME

If an enterprise's reasoning, memory, and institutional habits are mediated by a provider-controlled agentic runtime, the provider becomes the de facto operator of operational intelligence. The enterprise keeps its legal shell, but the economic substance of decision-making migrates into an agent layer it does not control.

The historical analog is familiar: enterprises that built critical workflows on a single cloud provider discovered switching costs only when that provider changed terms. Agentic AI compresses the same dynamic, because operational intelligence – not just compute – is what is at stake.

## Concentration risk map

### Pricing power

Leaving is expensive because memory and workflows live in the runtime.

### Strategic withdrawal

Use cases, regions, or industries can lose support.

### Ownership changes

Partnerships and incentives can change faster than enterprise systems migrate.

### Silent defaults

Retention, logging, routing, and memory settings drift over time.

# Move from provider trust to an enforceable runtime contract.

The answer is not to reject frontier providers or managed agents. The answer is to keep architectural authority over scope, memory, tools, and publication inside the enterprise before any third-party agent is allowed to act.

## HERMETIC KNOWLEDGE ISOLATION - V1

### GATEWAY

#### Choose one active domain

The edge selects and signs the active domain for each runtime operation – before retrieval, memory, tools, or model calls begin. Ambiguity is closed at the boundary, not delegated.

SIGNED SCOPE  
ENVELOPE

FAIL-CLOSED ON NULL  
DOMAIN

### RUNTIME

#### Preserve labels everywhere

Artifacts, cache entries, memories, embeddings, tool calls, async jobs, and outputs carry their provenance and scope label from creation to consumption. No helper subsystem can erase it.

LABEL-PRESERVING  
SURFACES

EXACT-DOMAIN  
EQUALITY

### PUBLICATION

#### Make sharing explicit

Cross-domain knowledge moves only through reviewed, auditable publication that creates a new domain-local copy – never through silent fallback, global scope, or inferred visibility.

DELIBERATE  
REPLICATION

AUDITABLE PROVENANCE

Every artifact belongs to one domain. Every runtime operation runs in one active domain. Cross-domain answers are composed through explicit publication, handoff, or audited joins - never silent global fallback.

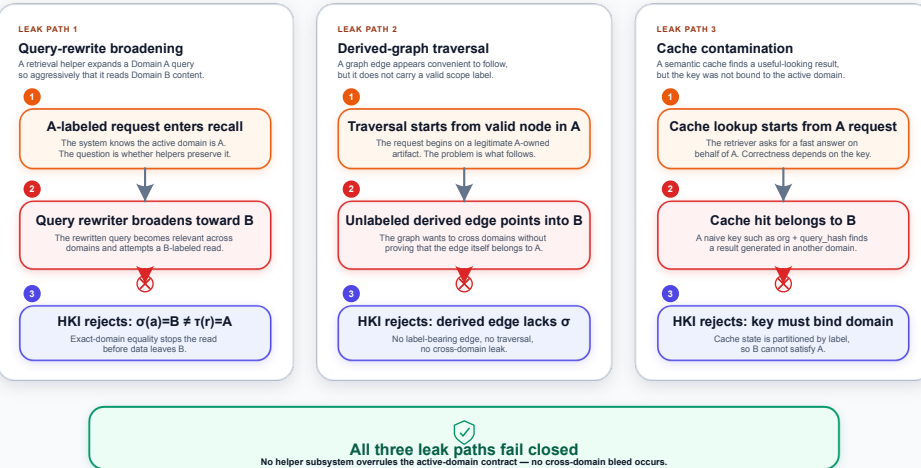


# Security becomes credible when the boundary is falsifiable.

HKI is not a slogan. It is a set of runtime checks that can be attacked, probed, and released with evidence.

## Three leak paths, all blocked

HKI does not rely on good intentions from helper subsystems.  
Each attempted boundary violation is rejected against the active-domain contract.



The recurring failures are query-rewrite broadening, derived graph bleed, and cache contamination. HKI blocks all three for the same reason: the system must preserve the active domain through every transformation, not only at the final read.

- Missing scope is rejected.
- Unauthorized active domains are rejected.
- Cross-domain graph edges are not traversed.
- Cache keys cannot omit the active domain.
- Publication creates new domain-labeled copies.

# Build the boundary while it still belongs to you.

No enterprise adopts runtime custody in one cutover. The practical path is a staged migration from inventory to enforcement to ongoing evidence.

## NEAR-TERM ROLLOUT SEQUENCE

### 01 AUDIT

Find null-scoped artifacts, weak cache keys, and endpoints that answer without an explicit active domain. Map every surface that touches enterprise knowledge.

### 02 LABEL

Ensure every artifact type – documents, embeddings, memory entries, cache slots – carries exactly one domain identity. Eliminate implicit global labels.

### 03 PROPAGATE

Select and sign the active domain at the request edge. Downstream services reject requests that carry a conflicting or absent scope claim.

### 04 ENFORCE

Turn null-scope reads and label-dropping paths into test failures and release blockers. The boundary must be falsifiable – claims are not enough.

### 05 EVIDENCE

Add observability, adversarial probes, and release proofs that the boundary held across all surfaces. Conformance is the measurable output.

## BOTTOM LINE

# Isolation cannot remain a provider promise. It has to become architecture.

The enterprises that remain sovereign in the agentic era will define what belongs where, what can be retrieved, what can be remembered, what can be shared, and what must be explicitly published before it crosses a domain boundary.

This document introduces **Hermetic Knowledge Isolation (HKI)** — a proposed open framework for enforcing that boundary at every runtime surface: gateway, retrieval, memory, tools, cache, async jobs, and telemetry. It is not a product. It is a contract any agentic platform can be built or audited against.

## THE FINAL TEST

If a runtime operation can answer without an active domain, the boundary is a claim. If it cannot, the boundary is architecture.

- ✓ One active domain signed at the gateway edge before any retrieval, memory, or tool call begins
- ✓ Labels preserved through memory, cache, embeddings, async jobs, and telemetry – no surface can drop them
- ✓ Cross-domain sharing only through deliberate, auditable publication – never silent fallback or inferred global scope
- ✓ Every boundary testable through adversarial probes and verifiable release evidence across all runtime surfaces

## WHERE TO START

Audit one runtime operation end-to-end. Ask: which domain is active, where is that signed, and what would change if it were absent? If nothing breaks, the boundary isn't real yet.